

RAG Chatbot Project Structure and Technology Explanation

Project Overview

This project is a Retrieval-Augmented Generation (RAG) AI chatbot developed using:

- React.js frontend
- FastAPI backend
- LangChain framework
- ChromaDB vector database
- Hugging Face embeddings
- Ollama
- LLaMA 3.2 model
- SSH deployment on Linux server

The chatbot allows users to:

- Upload PDF documents
- Ask questions from uploaded documents
- Retrieve accurate contextual answers
- View source pages from documents

The system combines Large Language Models with semantic document retrieval to create an intelligent AI assistant.

Complete Project Structure

```
/home/hafsa/
|
├── rag-chatbot/
|   |
|   ├── backend.py
|   ├── chroma_db/
|   ├── documents/
|   ├── venv/
|   ├── requirements.txt
|   └── uploads/
|
├── chatbot-frontend/
|   |
|   ├── src/
|   |   ├── App.js
|   |   ├── App.css
|   |   └── index.js
```

```
| |
| |
| |└─ public/
| |└─ package.json
| |└─ build/
```

Frontend Explanation

Technology Used

- React.js
- Axios
- CSS

Purpose of Frontend

The frontend provides the graphical user interface for users to:

- Chat with AI
- Upload PDFs
- View responses
- View source citations

Main Frontend Files

App.js

This is the main React component responsible for:

- Managing chat messages
- Sending API requests
- Uploading files
- Displaying AI responses
- Handling Enter key events
- Showing loading states

App.css

This file handles:

- Styling
- Glassmorphism UI
- Bubble chat design
- Responsive layout
- Purple neon AI theme

Axios Usage

Axios was used to send HTTP requests from React frontend to FastAPI backend.

Installed Using

```
npm install axios
```

Why Axios Was Used

Axios provides:

- Faster API communication
- Easy JSON handling
- Better async support
- Cleaner frontend-backend integration

Where Axios Was Used

Inside App.js:

```
axios.post("http://localhost:8000/chat")
```

and:

```
axios.post("http://localhost:8000/upload")
```

Backend Explanation

Technology Used

- FastAPI
- Python
- LangChain
- ChromaDB
- Hugging Face
- Ollama
- LLaMA 3.2

Purpose of Backend

The backend performs:

- PDF loading

- Text extraction
 - Text chunking
 - Embedding generation
 - Vector storage
 - Semantic retrieval
 - AI answer generation
-

FastAPI Explanation

What is FastAPI?

FastAPI is a Python backend framework used to build APIs.

Why FastAPI Was Used

FastAPI provides:

- High speed
- Async processing
- API endpoint management
- Easy frontend integration
- Automatic Swagger documentation

Installed Using

```
pip install fastapi uvicorn
```

Where It Was Used

Inside backend.py:

```
app = FastAPI()
```

API endpoints:

```
@app.post("/chat")
```

```
@app.post("/upload")
```

LangChain Explanation

What is LangChain?

LangChain is a framework used to build AI applications using Large Language Models.

It helps connect:

- LLMs
- vector databases
- document loaders
- memory
- retrieval pipelines

Why LangChain Was Used

LangChain was used because it simplifies:

- RAG pipeline development
- PDF processing
- retrieval systems
- conversational memory
- vector search integration

Without LangChain, building the entire pipeline manually would be difficult.

Installed Using

```
pip install langchain
pip install langchain-community
pip install langchain-text-splitters
```

Where LangChain Was Used

PDF Loading

```
from langchain_community.document_loaders import PyPDFLoader
```

Text Splitting

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

Vector Database

```
from langchain_community.vectorstores import Chroma
```

LLM Connection

```
from langchain_community.llms import Ollama
```

RAG Chain

```
from langchain.chains import ConversationalRetrievalChain
```

Memory

```
from langchain.memory import ConversationBufferMemory
```

Hugging Face Explanation

What is Hugging Face?

Hugging Face provides pre-trained AI models for NLP and embeddings.

Why Hugging Face Was Used

Hugging Face was used to generate embeddings for document chunks.

Embeddings convert text into numerical vectors that can be understood by the vector database.

Without embeddings:

- semantic search would not work
- RAG retrieval would fail
- document understanding would not be possible

Installed Using

```
pip install sentence-transformers
```

Model Used

```
sentence-transformers/all-MiniLM-L6-v2
```

Why This Model Was Chosen

This model was selected because:

- lightweight
- fast
- good semantic accuracy
- suitable for Linux servers
- low RAM usage

Where Hugging Face Was Used

Inside backend.py:

```
from langchain_community.embeddings import HuggingFaceEmbeddings
```

Embedding creation:

```
embedding_model = HuggingFaceEmbeddings(  
    model_name="sentence-transformers/all-MiniLM-L6-v2"  
)
```

Benefits Provided

Hugging Face embeddings improved:

- semantic understanding
- contextual retrieval
- document search accuracy
- chatbot response quality

ChromaDB Explanation

What is ChromaDB?

ChromaDB is a vector database.

It stores embeddings generated from documents.

Why ChromaDB Was Used

ChromaDB was used because:

- it is lightweight
- easy to integrate
- optimized for vector search
- ideal for RAG systems

Installed Using

```
pip install chromadb
```

Where ChromaDB Was Used

Inside backend.py:

```
from langchain_community.vectorstores import Chroma
```

Database creation:

```
vector_db = Chroma.from_documents(  
    documents=texts,  
    embedding=embedding_model,  
    persist_directory=DB_PATH  
)
```

Retriever:

```
retriever = vector_db.as_retriever(  
    search_kwargs={"k": 4}  
)
```

Benefits Provided

ChromaDB helped:

- store document embeddings
 - retrieve relevant chunks
 - improve answer relevance
 - enable semantic search
 - make large PDF querying possible
-

Ollama Explanation

What is Ollama?

Ollama is a local LLM runtime used to run Large Language Models on Linux.

Why Ollama Was Used

Ollama was used because:

- it allows local AI execution
- no cloud API was required
- better privacy
- direct Linux deployment
- easy LLaMA integration

Installed Using

```
curl -fsSL https://ollama.com/install.sh | sh
```

Where Ollama Was Used

Inside backend.py:

```
from langchain_community.llms import Ollama
```

LLM initialization:

```
llm = Ollama(  
    model="llama3.2",  
    base_url="http://127.0.0.1:11434"  
)
```

LLaMA 3.2 Explanation

What is LLaMA?

LLaMA is a Large Language Model developed by Meta.

Why LLaMA Was Used

LLaMA 3.2 was used because:

- advanced reasoning
- conversational ability
- strong contextual understanding
- compatibility with Ollama

Model Pull Command

```
ollama pull llama3.2
```

Role in Chatbot

LLaMA generates final AI responses using:

- retrieved document chunks
 - user question
 - conversational context
-

RAG Pipeline Explanation

What is RAG?

RAG stands for:

Retrieval-Augmented Generation.

How RAG Works in This Project

Step 1

User uploads PDF.

Step 2

PDF text is extracted.

Step 3

Text is split into chunks.

Step 4

Hugging Face converts chunks into embeddings.

Step 5

Embeddings are stored in ChromaDB.

Step 6

User asks question.

Step 7

Retriever searches relevant chunks.

Step 8

Relevant chunks are sent to LLaMA.

Step 9

LLaMA generates contextual answer.

Memory System Explanation

Technology Used

ConversationBufferMemory

Purpose

Maintains conversation history.

Allows chatbot to:

- remember previous questions
- maintain context
- support conversational flow

Where Used

```
memory = ConversationBufferMemory(  
    memory_key="chat_history",  
    return_messages=True,  
    output_key="answer"  
)
```

Linux Deployment Explanation

Why Linux Was Used

Linux provides:

- better AI support
- server environment
- efficient resource usage
- SSH remote access

Tools Used

- PuTTY
- WinSCP
- SSH tunneling

SSH Tunneling Purpose

Used to:

- expose frontend/backend ports
 - access Linux server locally
 - bypass network restrictions
-

Key Features of Final Chatbot

- PDF upload support
 - RAG-based retrieval
 - Source citations
 - Semantic document search
 - AI conversational memory
 - Modern React UI
 - Linux deployment
 - Full-stack integration
 - Real-time chatbot interaction
-

Conclusion

This project successfully implemented a complete Retrieval-Augmented Generation chatbot using modern AI technologies.

The system combines:

- semantic retrieval

- vector databases
- large language models
- frontend-backend communication
- Linux deployment

to create an intelligent AI assistant capable of answering questions from uploaded documents with contextual accuracy.

The integration of LangChain, Hugging Face embeddings, ChromaDB, and LLaMA significantly improved document understanding, retrieval quality, and conversational performance.